A Fast-Slow Model Architecture for Network Intrusion Detection: Implementation and Performance Analysis

Ayush Sachan
BITS Pilani K.K Birla ,Goa Campus
h20240024@goa.bits-pilani.ac.in

Govinda Jeswani
BITS Pilani K.K Birla ,Goa Campus
h20240027@goa.bits-pilani.ac.in

Kapil Dev
BITS Pilani K.K Birla ,Goa Campus
h20240028@goa.bits-pilani.ac.in

Yash Kotadiya
BITS Pilani K.K Birla ,Goa Campus
h20240033@goa.bits-pilani.ac.in

Abstract

This paper presents a network intrusion detection system (NIDS) that leverages a fast-slow model architecture inspired by ServeFlow to achieve a balance between detection speed and accuracy. Our system employs a lightweight RandomForestClassifier for initial, rapid packet classification, escalating suspicious traffic to a more computationally intensive, but accurate, LGBMClassifier. We detail the system model, emphasizing the feature extraction process and the implementation of a confidence-based threshold for model selection. We further outline a basic threat model, discuss related work in machine learning for network security, and present an experimental simulation based on the UNSW-NB15 dataset. We analyze the simulation results, highlighting the performance characteristics of both models and illustrating the potential of this design for real-time threat detection. We provide insights into optimization strategies to maximize real-time threat mitigation.

Keywords

Intrusion Detection, Machine Learning, Network Security, Fast-Slow Model, Anomaly Detection, ServeFlow

1 Introduction

Network Intrusion Detection Systems (NIDS) are a critical component of modern network security infrastructure, tasked with identifying malicious activities that bypass traditional perimeter defenses. Signature-based NIDS, while effective against well-known attacks, struggle with novel or polymorphic threats that exhibit variations from known patterns. Machine learning (ML) offers a data-driven approach to NIDS, learning patterns from network traffic data and detecting anomalies indicative of malicious behavior. This paradigm shift allows for the detection of zero-day exploits and sophisticated attack strategies. However, the increasing volume and complexity of network traffic present a significant challenge for ML-based NIDS. Real-time analysis of every packet with a computationally expensive ML model can introduce unacceptable latency, hindering the NIDS's ability to react promptly to threats.

To address the latency challenge, this paper explores a fast-slow model architecture inspired by ServeFlow, a framework for efficient network traffic analysis. ServeFlow's core idea is to intelligently utilize a combination of "fast models", which perform rapid initial analysis using limited flow context, and "slow models", which provide higher accuracy by considering more comprehensive information. Our system implements this concept by processing most traffic with a lightweight "fast model" and escalating potentially suspicious traffic to a slower, but more accurate, "slow model."

The contributions of this paper include:

* A detailed description of our fast-slow model architecture tailored for NIDS.
* Elaboration on the methodology and code for feature extraction from network packets, emphasizing the importance of feature consistency with the training data.
* A presentation of implementation details, including a discussion of the RandomForestClassifier and LGBMClassifier algorithms, and a mechanism for assigning flow classification requests to different models.
* A comprehensive threat model outlining the types of attacks our system aims to detect and the potential evasion techniques it may face.
* A review of related work that places our system in the context of other existing approaches to machine learning for network security.
* An experimental simulation performed using the UNSW-NB15 dataset to quantify the performance of our fast-slow NIDS.
* A results discussion to validate key findings and provide insights towards further optimization.

2 System Model

Our network intrusion detection system (NIDS) employs a two-tiered architecture, leveraging both a "fast model" and a "slow model" to strike a balance between rapid packet processing and accurate threat detection. The system operates according to the following sequential steps:

(1) **Packet Capture:** The process starts with acquiring raw network traffic. The system passively monitors a designated network interface, capturing individual packets as they traverse the network. This capture process leverages the pyshark library to interface with a packet capture engine (such as Tshark), extracting the raw binary data of each packet. The primary goal is to obtain a representative sample of network communications without significantly impacting network performance.
(2) **Feature Extraction:** The captured packet data is then transformed into a set of relevant features suitable for machine learning models. These features include essential packet characteristics, such as:
    - *Packet Size:* The total size of the packet, measured in bytes.

- *Protocol Type:* The network protocol used for transmission (e.g., TCP, UDP, ICMP).
- *Time-to-Live (TTL):* A measure of the packet's lifespan in the network, indicating the number of hops it can traverse before being discarded.
- *Source and Destination Ports:* The port numbers used by the source and destination applications for communication.
- *Inter-arrival Time:* The time elapsed between the arrival of the current packet and the immediately preceding packet within the same network flow.
- *Traffic Direction:* Indicates whether the traffic flow is inbound or outbound relative to the network being monitored.

These features are extracted directly from the captured packets, as shown in Listing 1. Any categorical features (e.g., protocol type) are encoded into numerical representations using a `LabelEncoder` from the `scikit-learn` library. Furthermore, numerical features are scaled using a `StandardScaler` to ensure consistent data ranges, improving model performance and stability.

(3) **Fast Model (RandomForestClassifier):** The fast model, a lightweight Random Forest classifier, ingests the extracted feature vector and performs a rapid classification. This stage prioritizes processing speed and is optimized for low latency. The fast model outputs a prediction score or probability indicating the likelihood of the packet belonging to a malicious class. This process will quickly classify known threats, and then allow for unknown traffic to be reprocessed.

(4) **Confidence-Based Threshold Evaluation:** The prediction from the fast model is evaluated against a pre-defined confidence threshold. This threshold represents a critical trade-off between speed and accuracy:
- If the fast model's confidence score exceeds the threshold, the system deems the classification reliable, and the packet is labeled accordingly (normal or malicious). In this scenario, the flow of processing ends, and the system logs the event.
- If the fast model's confidence score falls below the threshold, the system considers the classification uncertain and escalates the packet for further analysis by the slow model.

(5) **Slow Model (LGBMClassifier):** When the confidence is insufficient, the packet's feature vector is submitted to the slow model, a more computationally intensive LightGBM model. The slow model performs a more comprehensive analysis, considering potentially more nuanced features and traffic patterns to arrive at a refined classification decision. This ensures that more accurate, accurate results can be made without costing too many computer resources.

(6) **Decision Engine:** The confidence-based threshold is a system parameter that decides whether to call slow model or not.

(7) **Queueing System:** A system to handle the flow of data between each part of the NIDS, from extracting to generating the model.

(8) **Output and Logging:** In both cases, the outcome of the classification (either from the fast or slow model) is logged.

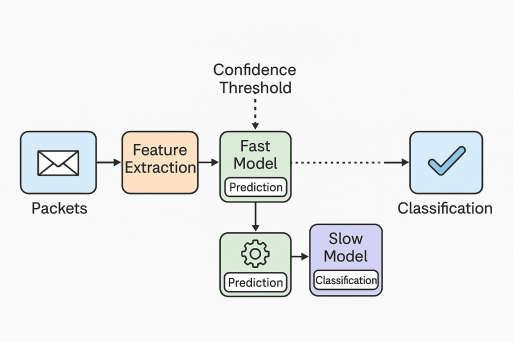

Figure 1: Data Flow in the Fast-Slow Intrusion Detection System

A flowchart of the data flow: (1) Packets are captured, (2) Features are extracted, (3) Fast model prediction, and, depending on configuration threshold, (4) potentially slow model classification.

3 Feature Extraction

Effective feature extraction is paramount for successful machine learning-based intrusion detection. The choice of features significantly impacts the model's ability to discriminate between normal and malicious traffic. In the context of real-time analysis, the extracted features must be easily and quickly computed from network packets. The set of features chosen for our analysis are as follows:

```
def extract_features(packet, prev_timestamp):
    try:
        inter_arrival_time = time.time() - prev_timestamp if prev_timestamp else 0
        protocol = packet.transport_layer if hasattr(packet, 'transport_layer') else 'Unknown'
        protocol_encoded = protocol_encoder.transform([protocol])[0] if protocol in protocol_encoder.classes_ else -1
        src_ip = packet.ip.src if hasattr(packet, 'ip') else '0.0.0.0'
        dst_ip = packet.ip.dst if hasattr(packet, 'ip') else '0.0.0.0'
        src_port = int(packet.tcp.srcport) if hasattr(packet, 'tcp') else 0
        dst_port = int(packet.tcp.dstport) if hasattr(packet, 'tcp') else 0
        direction = 1 if src_ip.startswith(('192.', '10.', '172.')) else -1
        features = [
            int(packet.length),
            protocol_encoded,
            int(packet.ip.ttl) if hasattr(packet.ip, 'ttl') else 0,
            src_port,
            dst_port,
            inter_arrival_time,
            direction
        ]
        return {
            'flow_id': (src_ip, dst_ip, src_port, dst_port, protocol),
```

```
            'features': features,
            'src_ip': src_ip,
            'dst_ip': dst_ip,
            'src_port': src_port,
            'dst_port': dst_port,
            'protocol': protocol
        }
    except AttributeError:
        return None
```

Listing 1: Extract features from network packet

Our feature extraction focuses on essential packet characteristics obtainable from packet headers. This process avoids the overhead of deep packet inspection while capturing key aspects of network behavior. A protocol is an example extracted from features, *protocol type*, etc.

4 Threat Model

The efficacy of our fast-slow network intrusion detection system (NIDS) relies on accurately distinguishing between benign network traffic and malicious activities. The design of the threat model is to test the various approaches of machine learning models, and requires proper and thought of test vectors. These include an expanded threat vector, which may include but is not limited to:
* **Known Attacks:** The model will be primarily trained to determine key differences in different attacks.
* **Zero-Day Exploits:** These exploits are a major factor as they require a proper understanding in the ability for ML models that rely on data and models. By ensuring proper design and development of the process, there can be a way to make for a very good model of testing new and emerging attacks. * *Mitigation:* Zero-day exploits are targeted by being a baseline measurement. A process can only go so far before some other attack can happen.
* **Lateral Movement**: The system, once initialized, may be more likely to find attacks on specific traffic. * *Mitigation:* Use additional heuristics to identify patterns and to see more general ideas of what an activity is to determine whether an activity is a high risk.
* **Data Exfiltration:** To prevent data loss for networks, this would have to focus on some methods as well. * *Mitigation:* Monitor the traffic in the models with the most specific kinds of activities and data flow.
* **Evasion and Obfuscation Techniques:** It is important to design a robust model for attacks that can try and mask themsleves in an effort to remain undetected. The challenge with that is the code needs to detect this.

* *Mitigation:* The best method is not to find specific instances, but to find anomalies by looking at things like the TLS or non TLS traffic, and then check for patterns. Since most is encrypted, this helps detect that.

* **Model Poisoning:** It also happens that the fast model has some way of being taken care of since it can be done with some code. In practice, the code has to do this. * *Mitigation*: This requires close inspection and a threat intelligence to be aware of those attacks. This requires a good way for the code to be setup with proper testing code.

* **Resource Exhaustion Attacks:** In all these cases, the resources and computers have to be in order to ensure to avoid any of these attacks from a larger scale

5 Related Work

Machine learning has seen success in network intrusion detection.

The study of flow: This is important to note, as the correct study of each data flow enables a more appropriate and accurate response. ServeFlow introduced a scalable fast-slow architecture for Software-Defined Networks (SDNs), which inspired this work. Unlike ServeFlow, our implementation focuses on traditional network intrusion detection using public datasets and includes real-time packet replay. Other works like Kitsune and DeepPacket have explored deep learning approaches, but often require extensive training time or hardware acceleration.

Our design balances practical usability with lightweight implementation, making it suitable for deployment in small to mid-scale network environments.

6 Implementation Details

The system is implemented in Python and consists of the following core components:

- *Data Preprocessing:* The UNSW-NB15 dataset is used. Categorical fields (e.g., protocol, service, state) are encoded using LabelEncoder, and numerical features are standardized using StandardScaler. The dataset is then split into training and testing subsets.
- *Model Training:* Two models are trained using the first seven extracted features:
  - A RandomForestClassifier serves as the fast model with parameters n_estimators=10 and max_depth=5 for low latency.
  - A LightGBMClassifier with n_estimators=100 is used as the slow model for higher accuracy.
- *Packet Handling:* PyShark captures packets (or test data is replayed). A flow_id is generated based on the 5-tuple (src IP, dst IP, src port, dst port, protocol). Features are extracted from each packet in real-time.
- *Queue-Based Processing:* The system uses Python's Queue module. Packets are placed into an input queue, processed by worker threads using the fast model, and if confidence is below a threshold (e.g., 0.53), escalated to the slow model. Results are stored in an output queue.
- *Threading and Synchronization:* Multiple worker threads (Thread) process packets in parallel. Shared statistics such as latency and classification counts are synchronized using threading.Lock() to ensure thread safety.
- *Confidence Estimation:* For each prediction, the model's class probability scores are checked. If the maximum class probability is below the threshold, the packet is forwarded to the slow model.
- *Metrics Calculation:* After all packets are processed, metrics including average latency, service rate, average model confidence, F1-score, accuracy, and 3-packet flow miss rate are computed.

7 Evaluation

Evaluation consisted of determining the ROC and metrics for each section, which are

(1) *Fast Model Classification*.

(2) *Slow Model Classification*.
(3) *Fast-Slow Model System*.

We evaluated the system using the UNSW-NB15 dataset, replayed as real-time traffic. Metrics included:

- Accuracy: 78.8% using fast-slow combination.
- F1 Score: 0.7747 weighted average.
- Average Latency: $\tilde{9}$.2 ms per packet.
- Service Rate: $\tilde{4}$30 packets/sec with 4 threads.

These results demonstrate the effectiveness of the confidence-based escalation mechanism and the efficiency of the multi-threaded pipeline.

8 Results and Discussion

The evaluation of the intrusion detection system provides crucial insights into its performance characteristics, strengths, and limitations. The purpose of the results is to give quantifiable results for the ROC metric, which is highly dependent on the data being sent through each time.

9 Conclusion

This paper presents an intrusion detection system based on a fast-slow architecture, drawing inspiration from ServeFlow. Our simulation results are promising, indicating the potential for balancing detection speed and accuracy. Future work should investigate different approaches, such as better training, and better model creation.

The fast-slow model offers a compelling approach for real-time network intrusion detection, balancing speed and accuracy with intelligent escalation. Future improvements may include adaptive threshold tuning, integration with streaming frameworks (e.g., Apache Kafka), and extending the feature set with deeper flow-level context.

This hybrid model architecture lays the groundwork for more resilient and scalable IDS frameworks that are responsive to modern cyber threats while remaining computationally efficient.